

Documentation technique

Pour reprendre le développement. Décrit l'architecture, les flux et les points d'extension. Le style du projet : français, modules courts, logique séparée de l'UI, tests unittest, pas de commit sans accord.

1. Vue d'ensemble

Application PyQt6 mono-processus. Une couche de logique pure (compilation, modèle de données, garde-fous, mode auteur) sans dépendance Qt, et une couche UI mince au-dessus. Le contenu pédagogique est de la donnée, pas du code : un parcours est un dossier.

<code>atelier_snake.py</code>	point d'entree, parse les arguments, choisit le parcours
v	
<code>fenetre.Fenetre</code>	UI, cable tout
├─ <code>modele_etape</code>	charge <code>parcours.json</code> + <code>meta.json</code> → objets <code>Etape</code> / <code>Parcours</code>
├─ <code>executeur</code>	compile et execute le C, rend une porte ouverte/fermee
├─ <code>tuteur_ia</code>	appelle claudie, applique garde_fous puis un filtre lexical
├─ <code>progression</code>	etat (exercices faits, crans debloques), persiste en json
├─ <code>coloration</code>	coloration syntaxique C
├─ <code>lsp_clangd</code>	diagnostics live (optionnel, degrade si clangd absent)
├─ <code>moodle_sync</code>	file d'evenements vers le compagnon LTI
├─ <code>gestion_niveaux</code>	mode auteur, via <code>dialogue_niveaux</code> + auteur
├─ <code>diagnostic</code>	chemins et presence des outils, via <code>dialogue_diagnostic</code>

2. Modèle de données

`modele_etape.py` définit deux dataclasses :

- `Etape` : un niveau. Champs lus depuis `meta.json` (`id`, `titre`, `type`, `mode`, `fichier_edite`, `cran_debloque`, `noeud_cours`, `sortie_attendue`, plus des champs pour les parcours projet). `dossier` pointe sur le répertoire du niveau.
- `Parcours` : la liste des `Etape` plus le `mode` (`isole` ou `projet`).

`charger_parcours_complet(dossier_contenu)` lit `parcours.json` (`ordre` + `mode`) et construit le `Parcours`. `chemins.contenu_racine(nom)` résout `contenu\<nom>`.

3. Portes et exécution

`executeur.py` compile et exécute du C, sans UI. Le code de sortie du programme fait foi.

- `_compiler_et_lancer(sources, includes, ...)` : compile avec `gcc` dans un dossier temporaire, exécute, renvoie un `Resultat(ok, sortie)`. Sur Windows, `_nom_binaire` ajoute `.exe`.
- `porte_programme(etape, code)` : mode « programme complet ». Compile le code, exécute, vérifie que la sortie contient les fragments de `sortie_attendue`.
- `porte_perso`, `porte_jalon`, `juger_test`, `porte_logique`, `construire_et_jouer_projet` : autres modes (test fourni, test à écrire, parcours projet Snake). Le Snake est parqué.

Le compilateur est appelé par le nom `gcc` : il doit être au PATH. `Atelier.bat` ajoute `w64devkit\bin` au PATH au démarrage.

4. Tuteur IA et garde-fous

`tuteur_ia.demander_aide(etape, code, question, niveau)` construit une consigne selon le cran (N0-N2 socratique, N3 borné) et appelle le moteur (`claud` ou `codex`) en sous-processus. Deux protections en sortie :

1. `garde_fous.masquer_si_solution(etape, reponse)` : garde-fou **structurel**. Extrait les blocs de code de la réponse, les rejoue contre la vraie porte de l'étape (`executeur.porte_programme` / `porte_perso`). Si un bloc, ou l'union des blocs emballée dans un `main`, ferait passer la porte, tout le code est remplacé par un refus, la prose reste. Bat la paraphrase et la fuite éparpillée.
2. Un filtre lexical ligne à ligne, en second rideau.

Limite connue : une fuite en prose pure (sans bloc de code) échappe au garde-fou.

Dans l'UI, l'appel tuteur tourne dans un `QThread` (`FilTuteur`) pour ne pas figer la fenêtre.

5. Progression et crans

`progression.py` porte l'état : exercices faits, cran d'aide disponible. Un exercice se débloque quand le précédent est validé ; franchir une porte peut débloquent un cran supérieur (`cran_debloque` du niveau). En mode démo, tout est ouvert et le cran est poussé à N3. L'état est persisté en JSON hors dépôt.

6. Mode auteur

Trois modules de logique pure, testables sans écran, et deux fenêtres.

- `gestion_niveaux.py` : `ajouter_niveau`, `retirer_niveau` (détache seulement, conserve le dossier), `reattacher_niveau`, `deplacer_niveau`, `lister_niveaux`, `dossiers_detaches`, et l'édition de contenu (`lire/ecrire_fichier_niveau`, `lire/ecrire_meta`). Écrit le même format JSON que le reste du dépôt.

- `auteur.py` : porte du mot de passe. Empreinte SHA-256, résolue depuis `auteur.json`, sinon la variable d'environnement `ATELIER_AUTEUR_MDP`, sinon un défaut.
- `reglages.py` : dernier parcours retenu, dans `reglages.json`.
- `dialogue_niveaux.py` : gestionnaire (ajouter, modifier, retirer, ordre, détachés) et éditeur de contenu à onglets.
- `dialogue_diagnostic.py` : fenêtre Emplacements et diagnostic.

Le tout est branché dans `fenetre._construire_menu` (menu Paramètres). L'édition du contenu recharge le parcours à la fermeture (`_recharger_parcours`).

7. Diagnostic et lancement

- `diagnostic.py` : `parcours_disponibles`, `chemins_cles`, `outils` (cherche gcc, clangd, claude dans le PATH puis dans `w64devkit\bin`), `ouvrir_dossier`.
- `atelier_snake._parcours_choisi` : priorité à `--parcours`, sinon `reglages.dernier_parcours()` (défaut `be_c`).
- `Atelier.bat` : détecte Python, ajoute le gcc portable au PATH, lance l'appli.

8. Moodle

`moodle_sync.py` gère l'appairage et une file locale d'événements vers le compagnon LTI (`compagnon\`, Flask, déployé séparément). Signale les portes franchies, rejoue la file au démarrage, expose le dernier score. Optionnel : l'appli marche sans connexion.

9. Tests

`tests\`, unittest. Les modules de logique pure se testent sans Qt ni gcc (`test_gestion_niveaux`, `test_reglages`, `test_diagnostic`, `test_modele_etape`, `test_progression`). Les tests de portes exigent `gcc`. Les tests du parcours projet (Snake) exigent SDL3, hors périmètre Windows.

```
python -m unittest discover -s tests
```

10. Points d'extension

- **Nouveau parcours** : un dossier sous `contenu\` avec `parcours.json`, il apparaît dans Changer de parcours. Aucun code à toucher.
- **Nouveau mode d'exercice** : ajouter un champ `mode` géré dans `executeur` et le branchement dans `fenetre._tester`.
- **Champs meta avancés dans l'éditeur** : `dialogue_niveaux.DialogueEdition` n'édite aujourd'hui que titre et sortie attendue ; les autres champs (`mode`, `cran_debloque`, `fichier_edite`, `noeud_cours`) s'ajouteraient là, en passant par `gestion_niveaux.ecrire_meta`.

- **Coloration dans l'éditeur de corrigé** : réutiliser `coloration.ColorationC` sur les éditeurs de `DialogueEdition`.

11. Conventions

Français dans les rendus, pas d'em-dash ni d'emoji. Modules courts, logique séparée de l'UI.

Fichiers locaux non commités : `auteur.json`, `reglages.json`, `progression.json`, `moodle_sync.json`, et le dossier `w64devkit\`. Pas de commit ni push sans accord.